Self-supervised Learning
Representation Learning

Hermann Blum

UNIVERSITÄT BONN

LAMARR INSTITUTE FOR MACHINE LEARNING AND ARTIFICIAL INTELLIGENCE

Robot Perception and Learning Lab
rpl.uni-bonn.de

Questions about the exam

120 min written exam

allowed material: 2 A4 sheets (both sides) of notes

needs to be hand written, not printed!

relevant content: all lectures and tutorials, except for July 24

consists of

- knowledge questions
- case studies
- coding / implementation (on paper)

Example Exam Question 1

Which of the following statements about DBSCAN is wrong?

- DBSCAN does not require a given number of clusters
- DBSCAN is a density based clustering
- DBSCAN scales efficiently with data
- DBSCAN can detect outliers

Example Exam Question 2

You are given a set of 2‘000‘000 images of plants and are tasked to categorize these into different species. You do not know how many species there are among the plants and you are not an expert on plant species. A colleague has agreed to label 2000 images for you. Describe a method to categorize the plants with high accuracy.

Example Exam Question 3

In the following template, implement the deep clustering algorithm.

```
# encoder: neural network mapping x to features

for x in minibatches(X):
    # x: input minibatch, shape (batch_size, input_dim)



for x in minibatches(X):
    # x: input minibatch, shape (batch_size, input_dim)



    loss =
    sgd(loss).backwards()
```

Feature Extraction

Goal: Map from X into a space Z that can be used as an ‚abstraction' of the data that is more convenient to process. E.g., points in Z often have a meaningful distance metric.

Feature Extraction

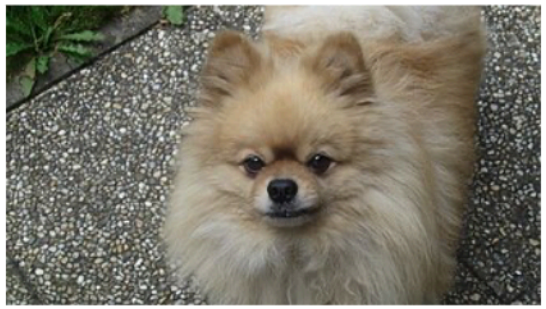
Goal: Map from X into a space Z that can be used as an ‚abstraction' of the data that is more convenient to process. E.g., points in Z often have a meaningful distance metric.



$\{a_1, a_2, a_3, \dots, a_N\}$



$\{b_1, b_2, b_3, \dots, b_N\}$



$\{c_1, c_2, c_3, \dots, c_N\}$

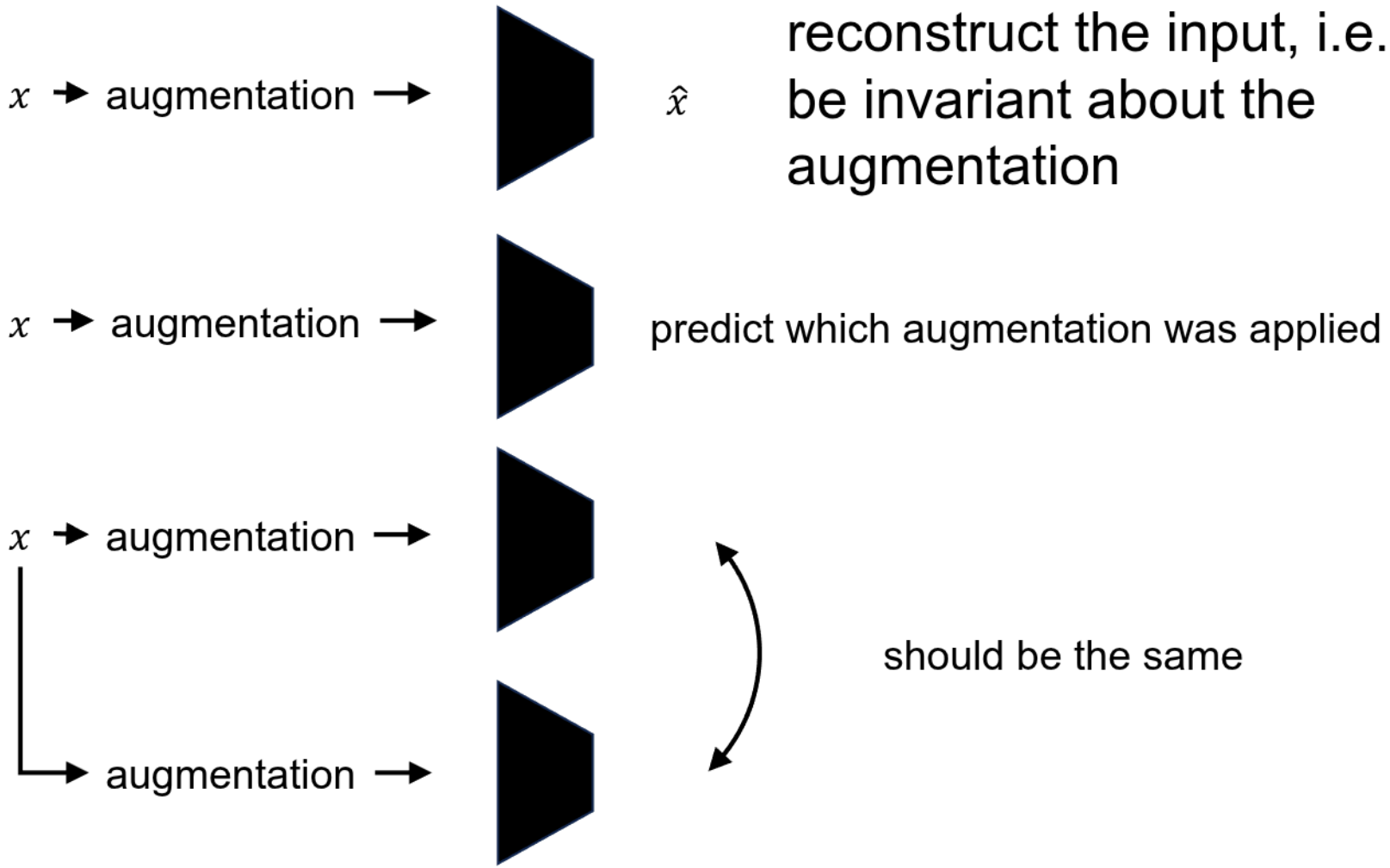
Euclidean Distance $d = \sqrt{(a-b)^2}$

Cosine Distance $d = \frac{a \cdot b}{|a| \cdot |b|}$

The next few lectures: Representation Learning

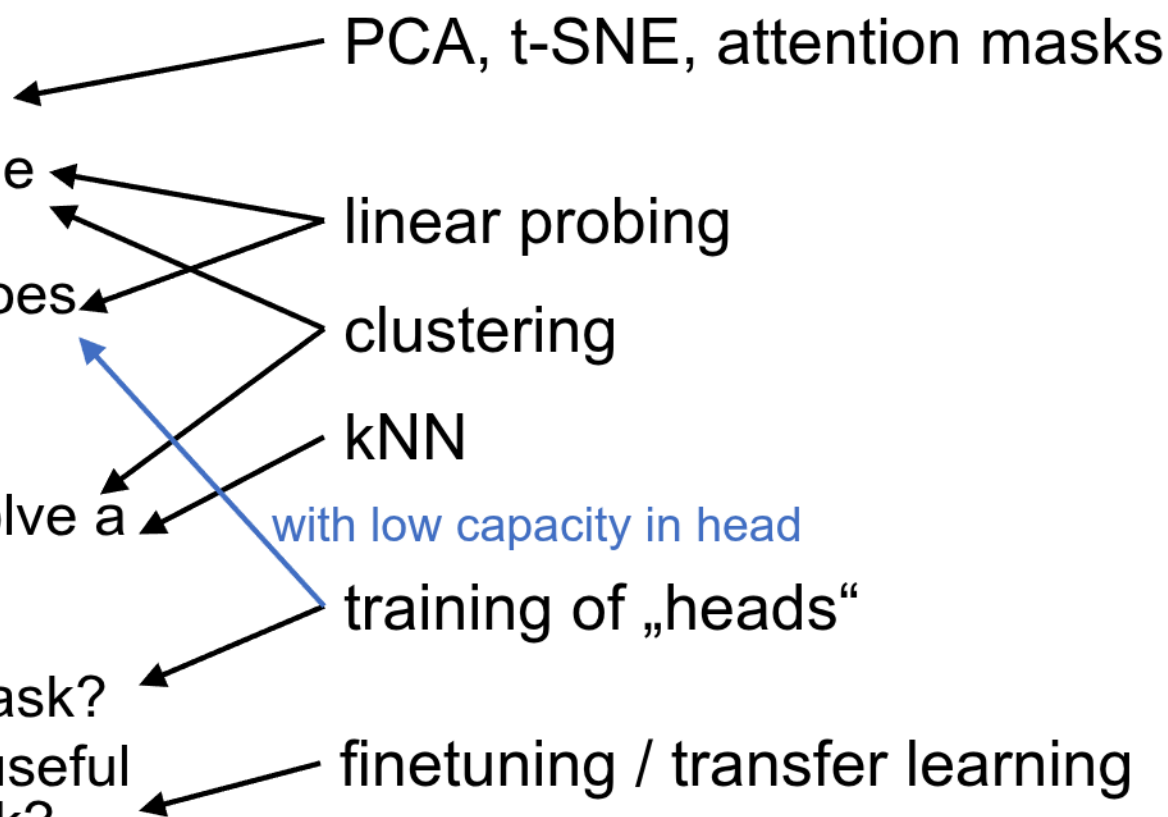
General Idea: Learn invariances or co-variances in the data to keep only relevant information in the representation.

Concept Comparison



How can we evaluate the quality of learned features?

- direct inspection
  - qualitatively through visualizations
  - To which changes in the input is the feature invariant?
  - With which changes in the input does the feature correlate?
- downstream applications
  - Can I directly use the feature to solve a task?
  - Is the feature useful as input to a (supervised) model for a defined task?
  - Are the SSL network parameters useful as pretraining for a supervised task?



SiamSiam

This CVPR 2021 paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Exploring Simple Siamese Representation Learning

Xinlei Chen Kaiming He
Facebook AI Research (FAIR)

Abstract

Siamese networks have become a common structure in various recent models for unsupervised visual representation learning. These models maximize the similarity between two augmentations of one image, subject to certain conditions for avoiding collapsing solutions. In this paper, we report surprising empirical results that ***simple Siamese*** *networks can learn meaningful representations even using* ***none*** *of the following: (i) negative sample pairs, (ii) large batches, (iii) momentum encoders. Our experiments show that collapsing solutions do exist for the loss and structure,*

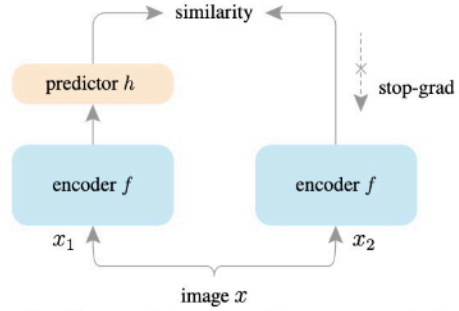
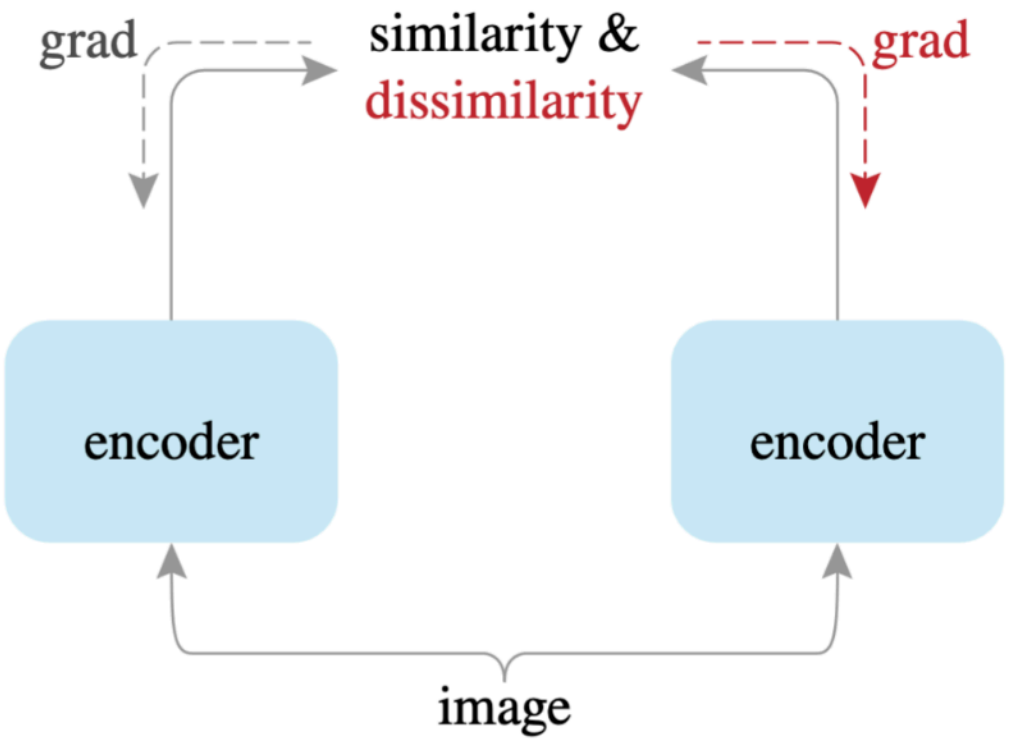
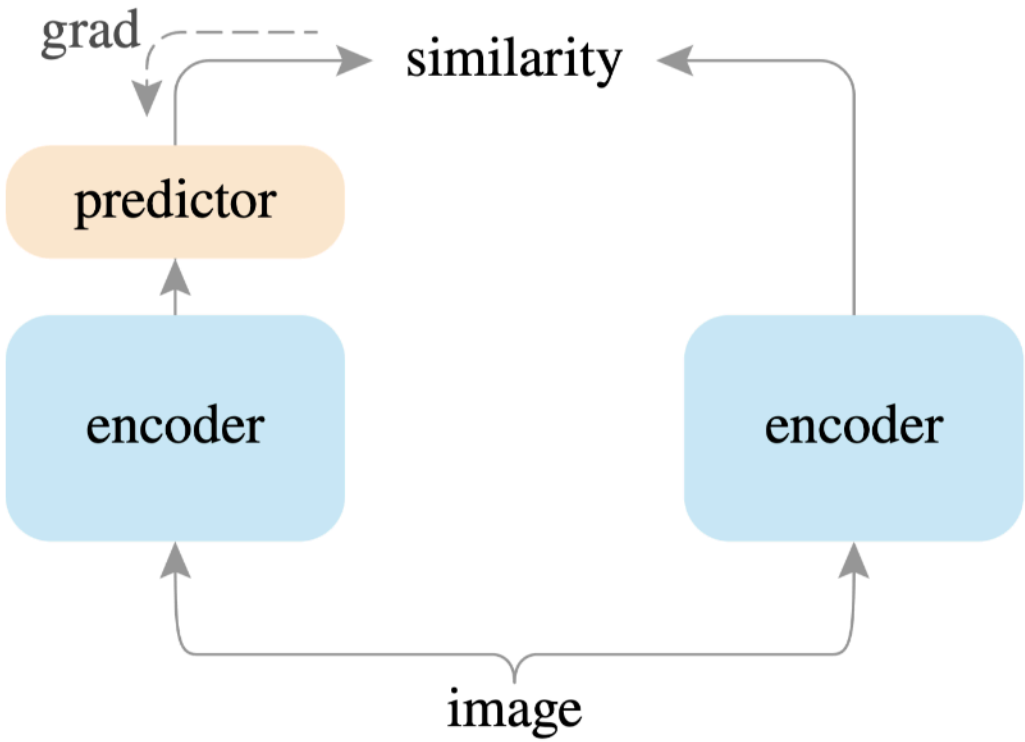


Figure 1. **SimSiam architecture**. Two augmented views of one image are processed by the same encoder network f (a backbone

Contrastive Learning without Negatives

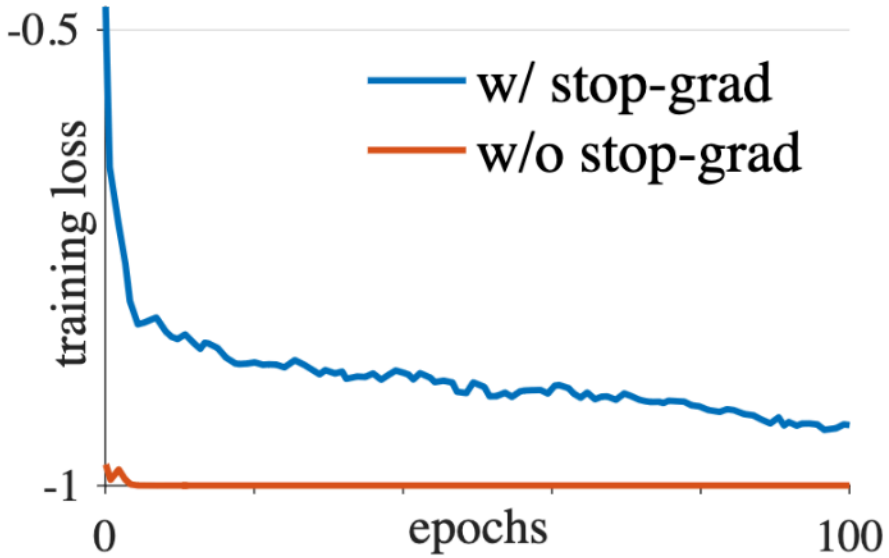


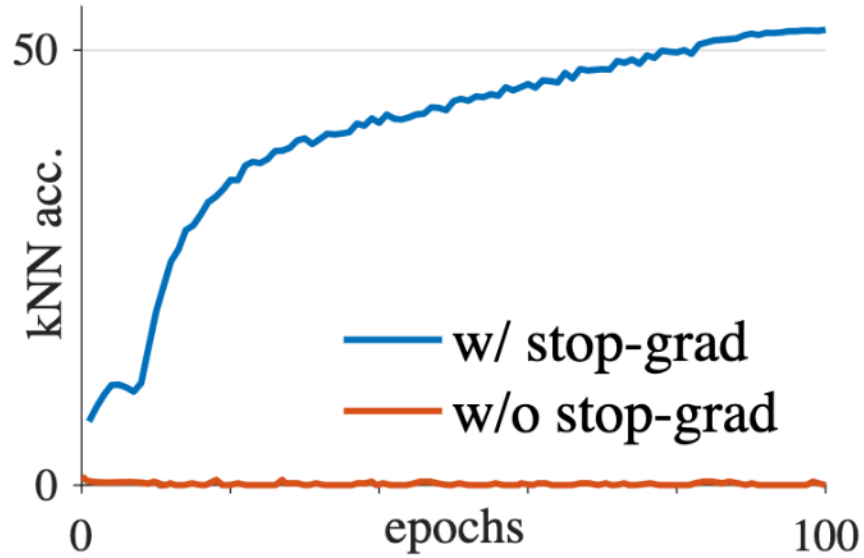
SimCLR



SimSiam

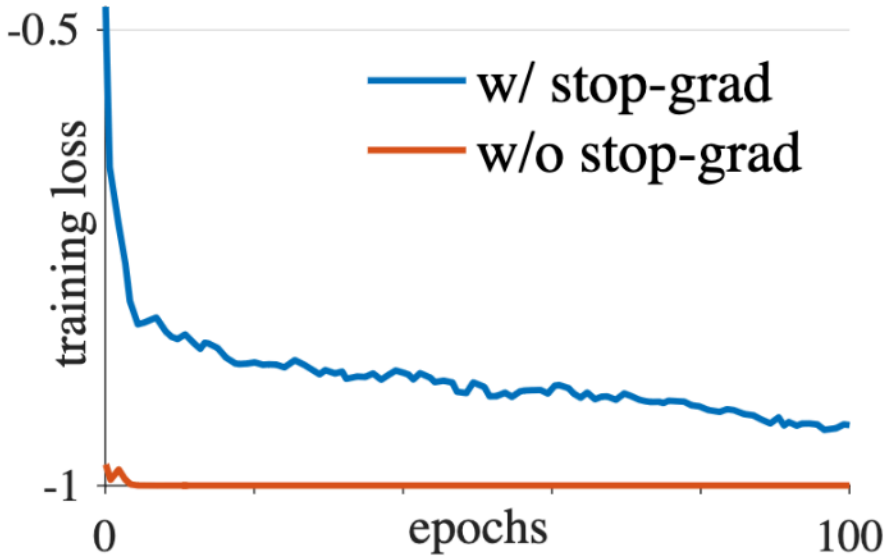
The Problem: Training Collapse

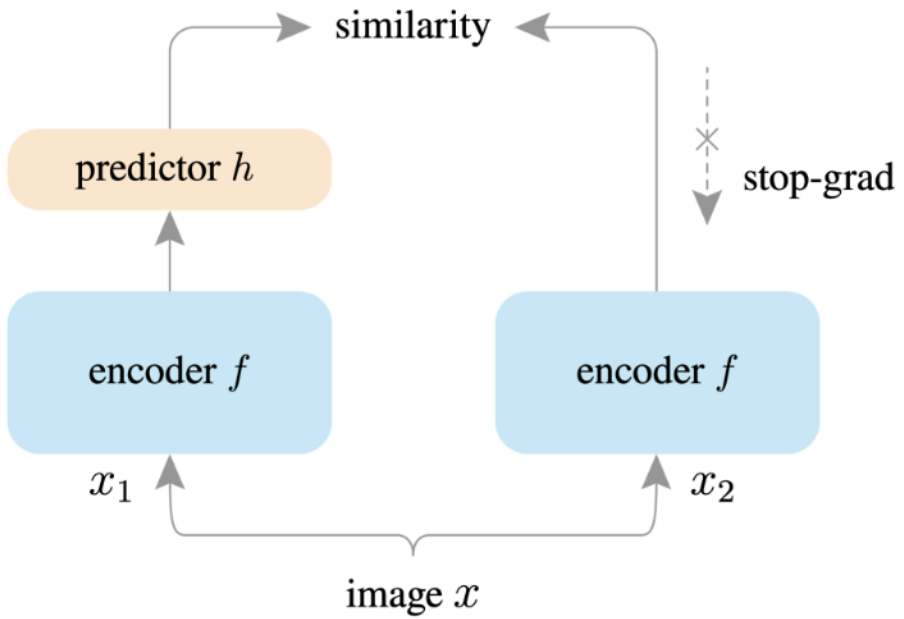




Different to supervised training: the loss does not directly model the downstream task, so a low loss is not always good

key finding: stop-grad with asymetric predictor avoids such collapse





The surprising finding: this works even without EMA teacher or huge batch size

Algorithm 1 SimSiam Pseudocode, PyTorch-like

```
# f: backbone + projection mlp
# h: prediction mlp

for x in loader: # load a minibatch x with n samples
   x1, x2 = aug(x), aug(x) # random augmentation
   z1, z2 = f(x1), f(x2) # projections, n-by-d
   p1, p2 = h(z1), h(z2) # predictions, n-by-d

   L = D(p1, z2)/2 + D(p2, z1)/2 # loss

   L.backward() # back-propagate
   update(f, h) # SGD update

def D(p, z): # negative cosine similarity
   z = z.detach() # stop gradient

   p = normalize(p, dim=1) # l2-normalize
   z = normalize(z, dim=1) # l2-normalize
   return -(p*z).sum(dim=1).mean()
```

forward pass through encoder and predictor

Algorithm 1 SimSiam Pseudocode, PyTorch-like

```
# f: backbone + projection mlp
# h: prediction mlp

for x in loader: # load a minibatch x with n samples
   x1, x2 = aug(x), aug(x) # random augmentation
   z1, z2 = f(x1), f(x2) # projections, n-by-d
   p1, p2 = h(z1), h(z2) # predictions, n-by-d

   L = D(p1, z2)/2 + D(p2, z1)/2 # loss

   L.backward() # back-propagate
   update(f, h) # SGD update

def D(p, z): # negative cosine similarity
   z = z.detach() # stop gradient

   p = normalize(p, dim=1) # l2-normalize
   z = normalize(z, dim=1) # l2-normalize
   return -(p*z).sum(dim=1).mean()
```

forward pass through encoder and predictor

symmetric loss: each branch can act as student or teacher

Algorithm 1 SimSiam Pseudocode, PyTorch-like

```
# f: backbone + projection mlp
# h: prediction mlp

for x in loader: # load a minibatch x with n samples
   x1, x2 = aug(x), aug(x) # random augmentation
   z1, z2 = f(x1), f(x2) # projections, n-by-d
   p1, p2 = h(z1), h(z2) # predictions, n-by-d

   L = D(p1, z2)/2 + D(p2, z1)/2 # loss

   L.backward() # back-propagate
   update(f, h) # SGD update

def D(p, z): # negative cosine similarity
   z = z.detach() # stop gradient

   p = normalize(p, dim=1) # l2-normalize
   z = normalize(z, dim=1) # l2-normalize
   return -(p*z).sum(dim=1).mean()
```

forward pass through encoder and predictor

symmetric loss: each branch can act as student or teacher

but teacher never propagates gradient

BERT

arXiv:1810.04805v2 [cs.CL] 24 May 2019

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

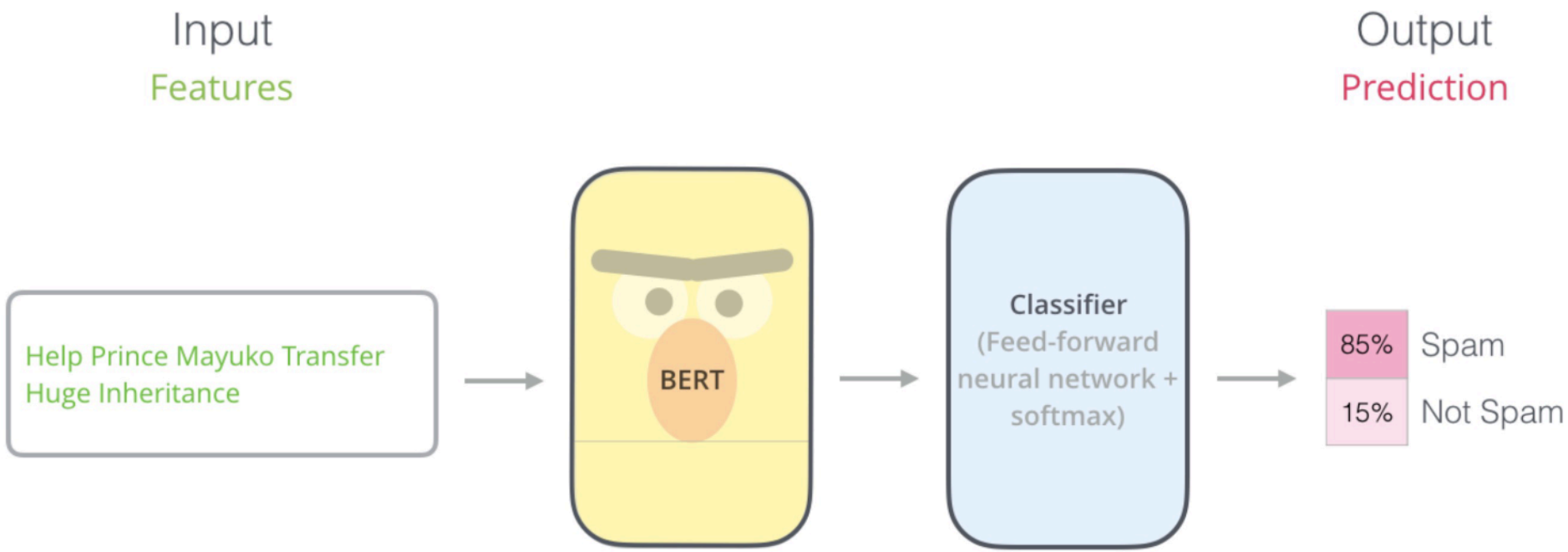
Jacob Devlin **Ming-Wei Chang** **Kenton Lee** **Kristina Toutanova**
Google AI Language
{jacobdevlin,mingweichang,kentonl,kristout}@google.com

Abstract

We introduce a new language representation model called **BERT**, which stands for **B**idirectional **E**ncoder **R**epresentations from **T**ransformers. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a re-

There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters et al., 2018a), uses task-specific architectures that include the pre-trained representations as additional features. The fine-tuning approach, such as the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018), introduces minimal task-specific parameters, and is trained on the

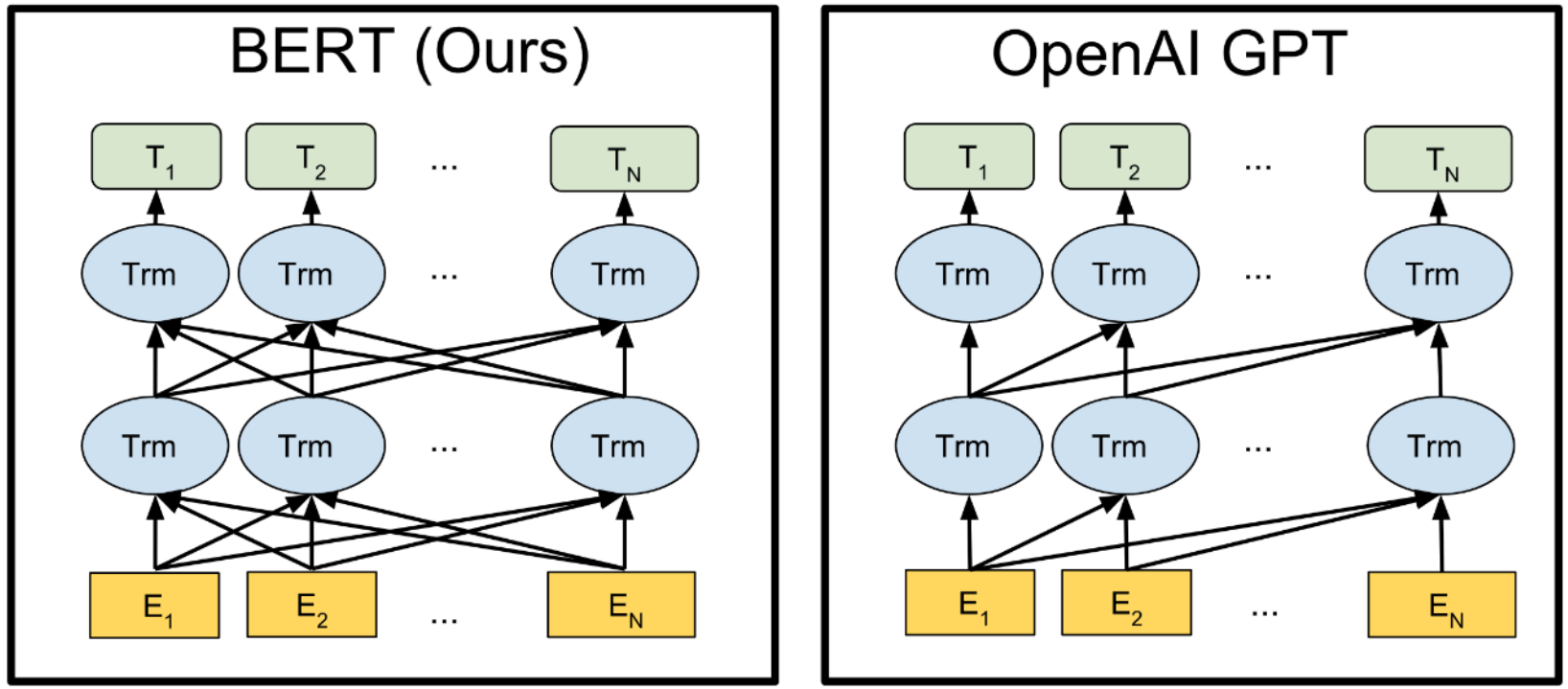
BERT example application: spam filtering



Output: representations of words based on their context

Jay Alammar, Illustrated BERT

BERT uses a bi-directional transformer

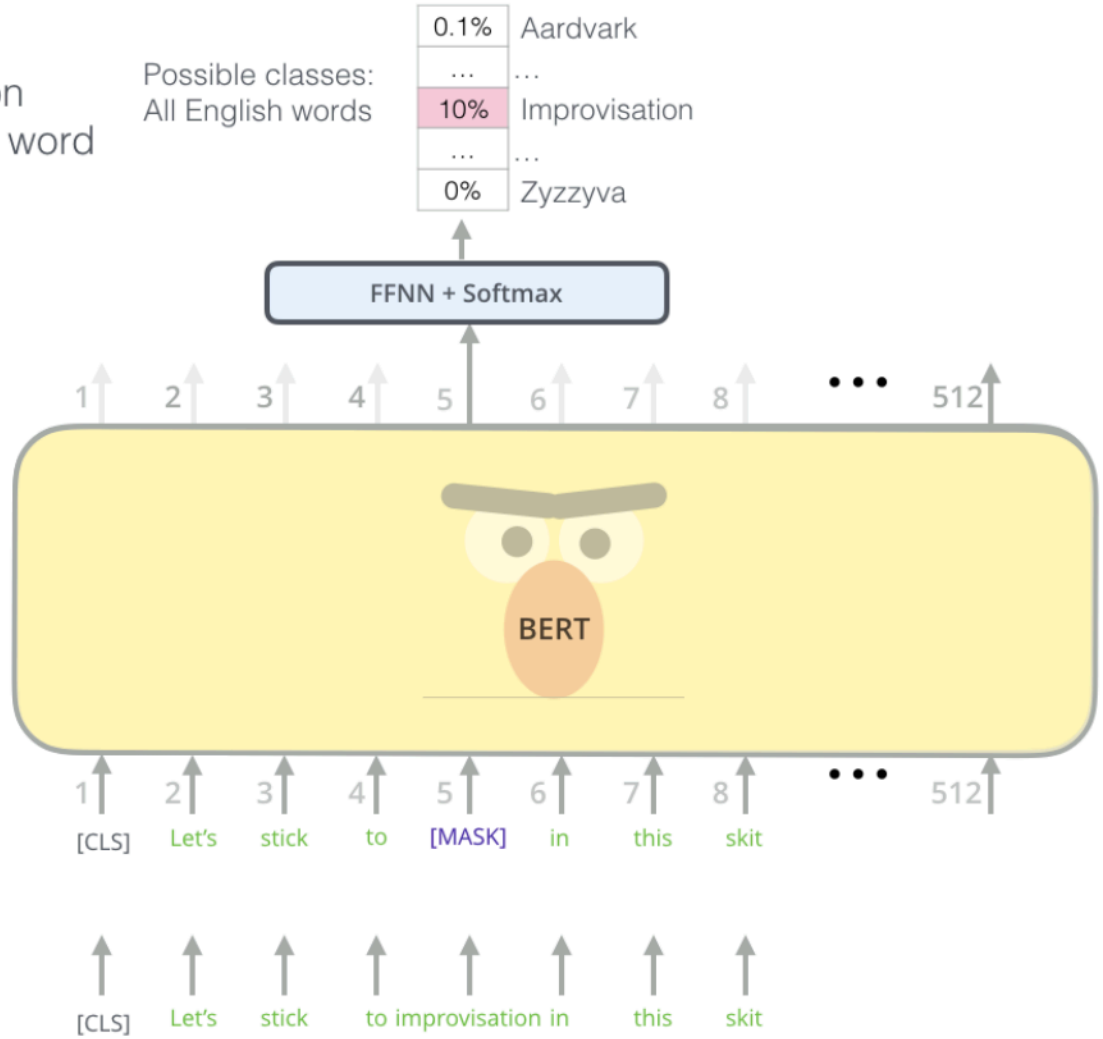


Output: representations of words based on their context

graphic by Milan Straka, Charles University

Pretrain Objective 1: Masked Word Prediction

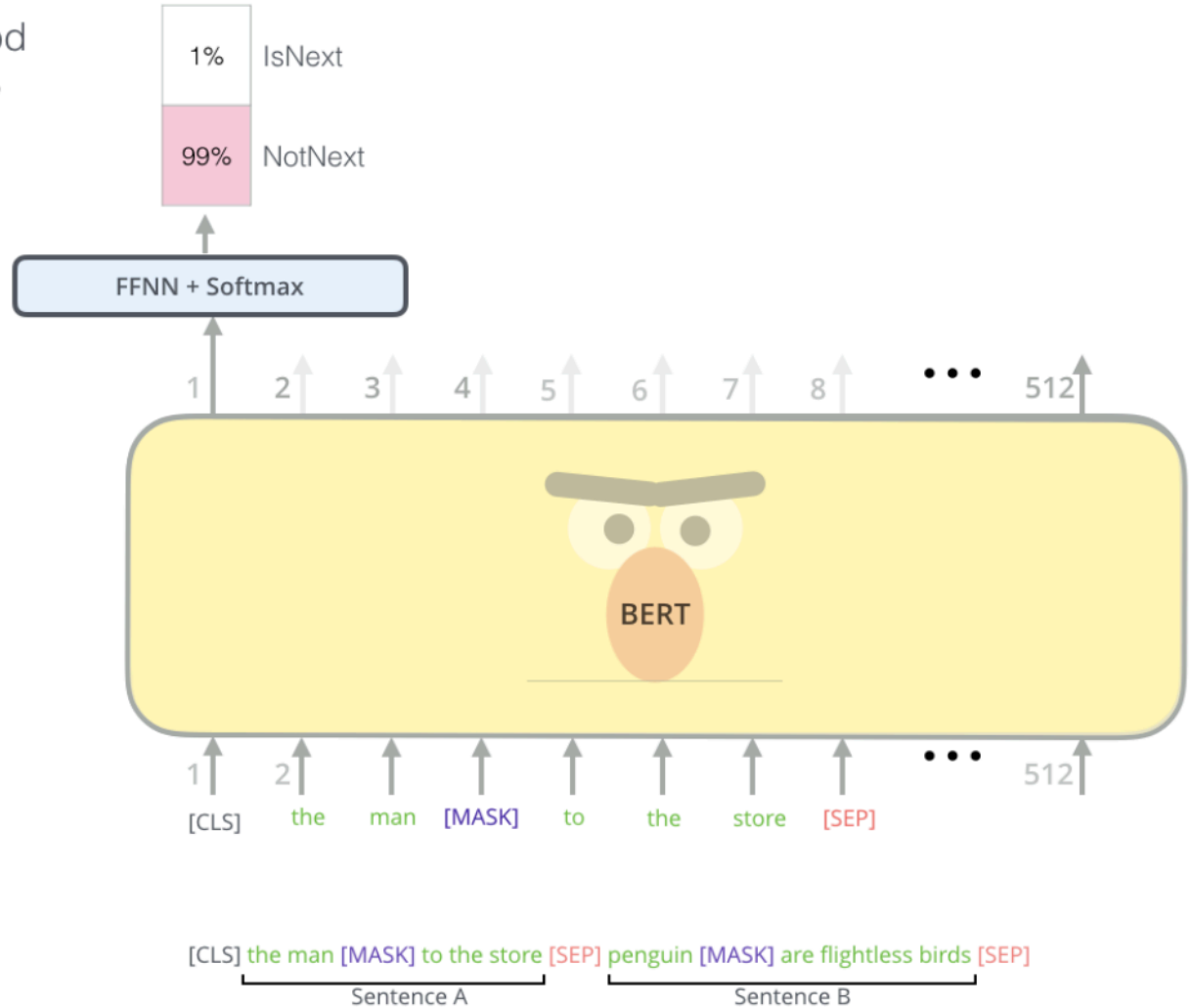
Use the output of the masked word's position to predict the masked word



Jay Alammar, Illustrated BERT

Pretrain Objective 2: Next Sentence Prediction

Predict likelihood that sentence B belongs after sentence A



Jay Alammar, Illustrated BERT

features can be finetuned or used directly

finetuned model to choose the most plausible continuation among 4 candidates

System	Dev	Test
ESIM+GloVe	51.9	52.7
ESIM+ELMo	59.1	59.2
OpenAI GPT	-	78.0
$\text{BERT}_{\text{BASE}}$	81.6	-
$\text{BERT}_{\text{LARGE}}$	**86.6**	**86.3**
Human (expert)†	-	85.0
Human (5 annotations)†	-	88.0

classifying whether a word is a name, based on frozen feature

System	Dev F1	Test F1
ELMo (Peters et al., 2018a)	95.7	92.2
CVT (Clark et al., 2018)	-	92.6
CSE (Akbik et al., 2018)	-	**93.1**
Fine-tuning approach		
$\text{BERT}_{\text{LARGE}}$	96.6	92.8
$\text{BERT}_{\text{BASE}}$	96.4	92.4
Feature-based approach ($\text{BERT}_{\text{BASE}}$)		
Embeddings	91.0	-
Second-to-Last Hidden	95.6	-
Last Hidden	94.9	-
Weighted Sum Last Four Hidden	95.9	-
Concat Last Four Hidden	96.1	-
Weighted Sum All 12 Layers	95.5	-

Table 7: CoNLL-2003 Named Entity Recognition results. Hyperparameters were selected using the Dev set. The reported Dev and Test scores are averaged over 5 random restarts using those hyperparameters.

DINO

This ICCV paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Emerging Properties in Self-Supervised Vision Transformers

Mathilde Caron[1,2] Hugo Touvron[1,3] Ishan Misra[1] Hervé Jegou[1]
Julien Mairal[2] Piotr Bojanowski[1] Armand Joulin[1]

[1] Facebook AI Research [2] Inria* [3] Sorbonne University

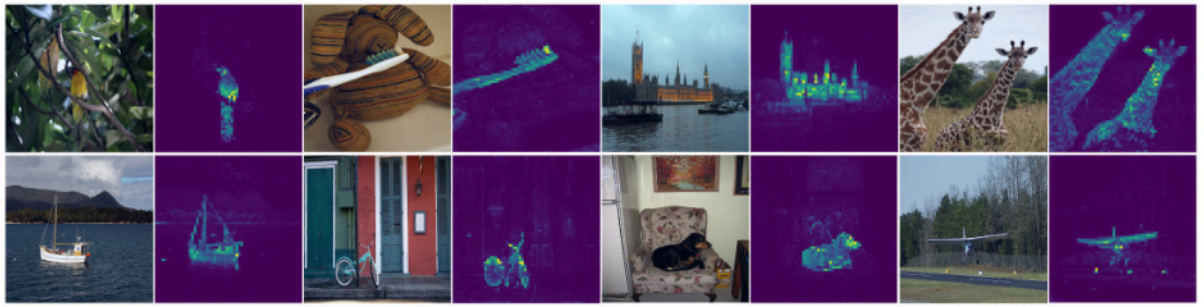
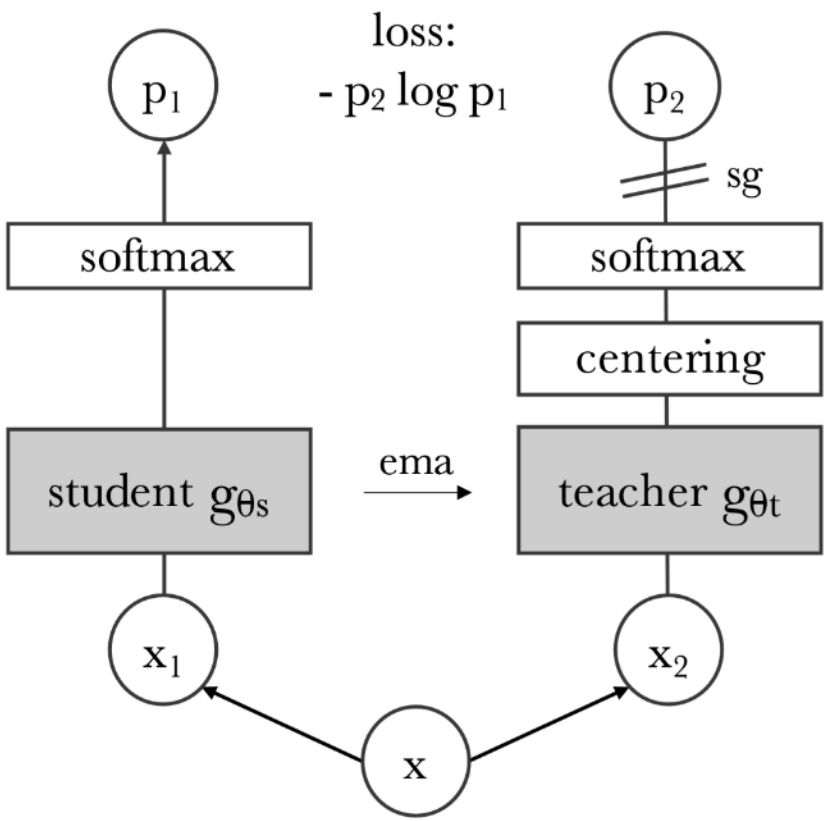


Figure 1: **Self-attention from a Vision Transformer with 8×8 patches trained with no supervision.** We look at the self-attention of the `[CLS]` token on the heads of the last layer. This token is not attached to any label nor supervision. These maps show that the model automatically learns class-specific features leading to unsupervised object segmentations.

DINO Overview



different loss to InfoNCE:

CE(softmax(z1), softmax(z2))

no negatives!

instead avoids collapse through centering in the teacher

$$c \leftarrow mc + (1-m)\underbrace{\frac{1}{B}\sum_{i=1}^{B} g_{\theta_t}(x_i)}_{\text{center of the current batch}},$$

EMA

DINO Overview

Multi-Crop

global crops for the teacher

global + local crops for the student

in addition: color jitter, gaussian blur, solarization

animation from Meta AI Blog

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

Generate multiple views of the same image. Then augment them.

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

Generate multiple views of the same image. Then augment them.

propagate views through teacher and student (teacher only global views)

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return -(t * log(s)).sum(dim=1).mean()
```

Generate multiple views of the same image. Then augment them.

propagate views through teacher and student (teacher only global views)

center + sharpen teacher

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

Generate multiple views of the same image. Then augment them.

propagate views through teacher and student (teacher only global views)

center + sharpen teacher

cross-entropy

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

Generate multiple views of the same image. Then augment them.

propagate views through teacher and student (teacher only global views)

center + sharpen teacher

cross-entropy

student applies SGD, teacher updates by EMA

Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

Generate multiple views of the same image. Then augment them.

propagate views through teacher and student (teacher only global views)

center + sharpen teacher

cross-entropy

student applies SGD, teacher updates by EMA

update the center (also EMA)

DINO works well with transformer architectures.

Method	Arch.	Param.	im/s	Linear	k-NN
Supervised	RN50	23	1237	79.3	79.3
SCLR [11]	RN50	23	1237	69.1	60.7
MoCov2 [13]	RN50	23	1237	71.1	61.9
InfoMin [54]	RN50	23	1237	73.0	65.3
BarlowT [66]	RN50	23	1237	73.2	66.0
OBoW [21]	RN50	23	1237	73.8	61.9
BYOL [23]	RN50	23	1237	74.4	64.8
DCv2 [9]	RN50	23	1237	75.2	67.1
SwAV [9]	RN50	23	1237	**75.3**	65.7
DINO	RN50	23	1237	**75.3**	**67.5**
Supervised	ViT-S	21	1007	79.8	79.8
BYOL* [23]	ViT-S	21	1007	71.4	66.6
MoCov2* [13]	ViT-S	21	1007	72.7	64.4
SwAV* [9]	ViT-S	21	1007	73.5	66.3
DINO	ViT-S	21	1007	**77.0**	**74.5**

classic ImgNet evaluation

with similar size, DINO works much better on ViT than on ResNet

DINO works well with transformer architectures.

Method	Arch.	Param.	im/s	Linear	k-NN
Supervised	RN50	23	1237	79.3	79.3
SCLR [11]	RN50	23	1237	69.1	60.7
MoCov2 [13]	RN50	23	1237	71.1	61.9
InfoMin [54]	RN50	23	1237	73.0	65.3
BarlowT [66]	RN50	23	1237	73.2	66.0
OBoW [21]	RN50	23	1237	73.8	61.9
BYOL [23]	RN50	23	1237	74.4	64.8
DCv2 [9]	RN50	23	1237	75.2	67.1
SwAV [9]	RN50	23	1237	**75.3**	65.7
DINO	RN50	23	1237	**75.3**	**67.5**
Supervised	ViT-S	21	1007	79.8	79.8
BYOL* [23]	ViT-S	21	1007	71.4	66.6
MoCov2* [13]	ViT-S	21	1007	72.7	64.4
SwAV* [9]	ViT-S	21	1007	73.5	66.3
DINO	ViT-S	21	1007	**77.0**	**74.5**

classic ImgNet evaluation

with similar size, DINO works much better on ViT than on ResNet

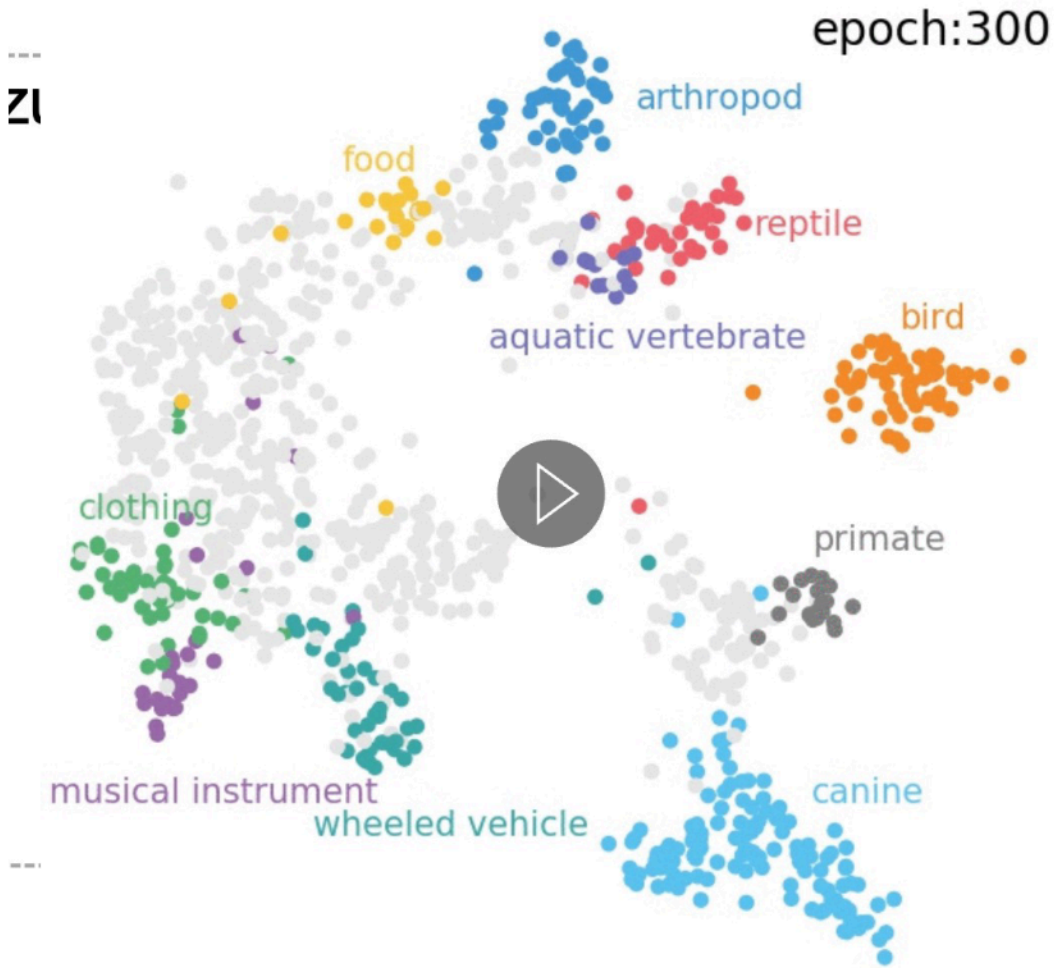
new eval: kNN directly with the features

> For finetuning evaluations, we initialize networks with the pretrained weights and adapt them during training. However, both evaluations are sensitive to hyperparameters, and we observe a large variance in accuracy between runs when varying the learning rate for example. We thus also evaluate the quality of features with a simple weighted nearest neighbor classifier (k-NN) as in [60]. We freeze

epoch: 0



What kind of evaluation is this?



from image-level loss emerges patch-level representation

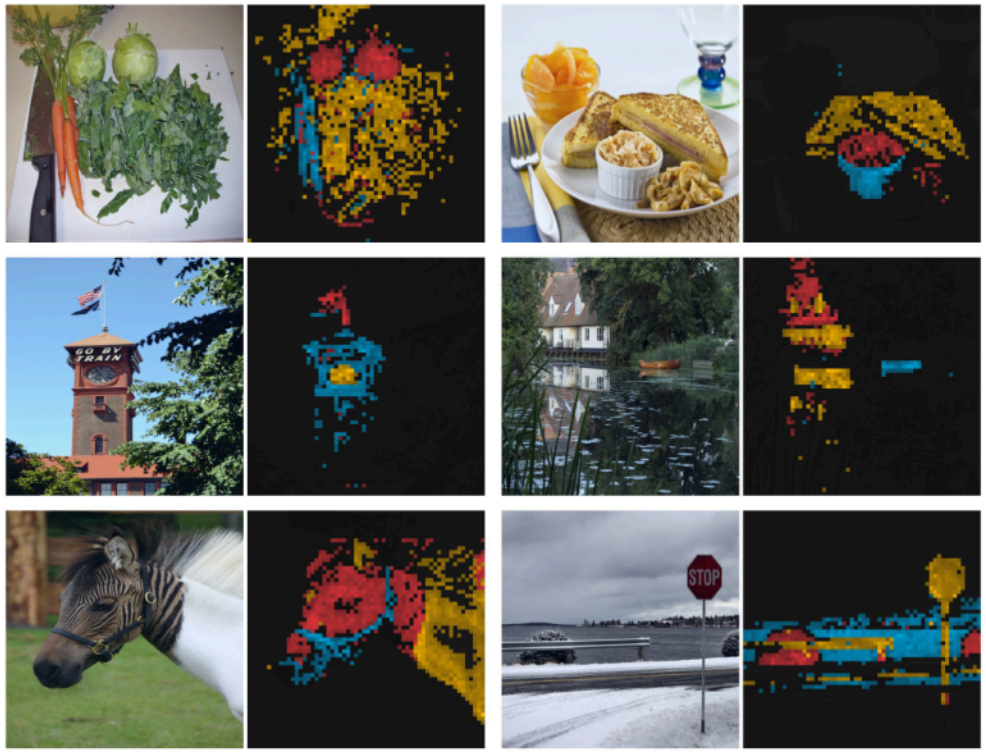
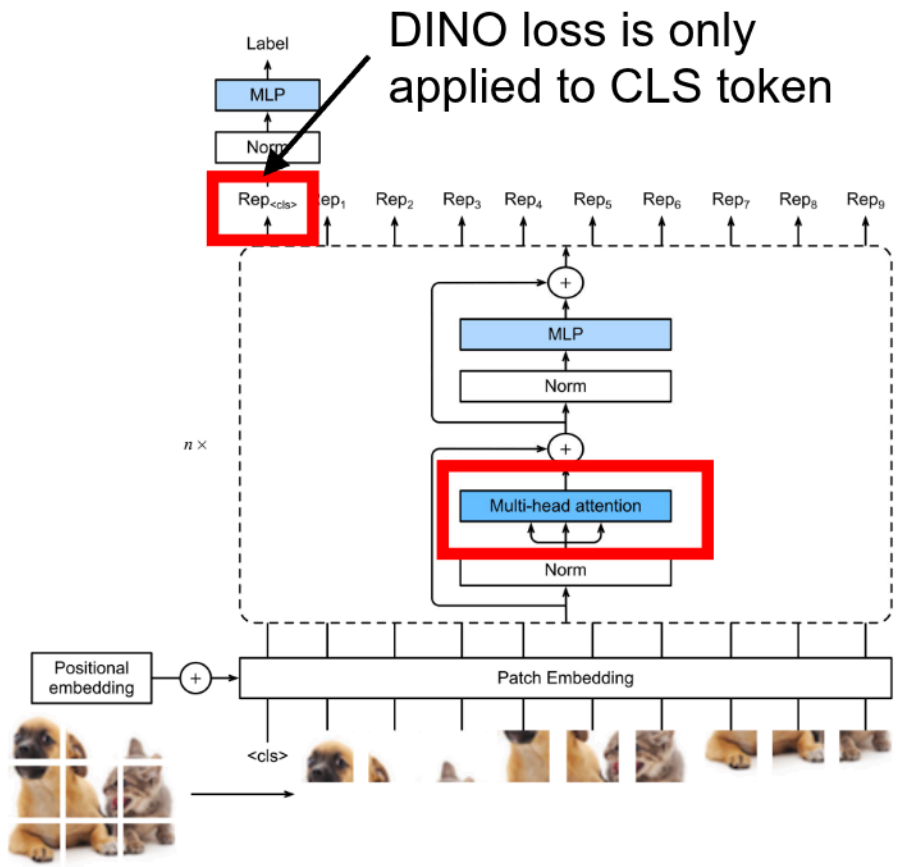


Figure 3: **Attention maps from multiple heads.** We consider the heads from the last layer of a ViT-S/8 trained with DINO and display the self-attention for `[CLS]` token query. Different heads, materialized by different colors, focus on different locations that represents different objects or parts (more examples in Appendix).



DINO v2

Published in Transactions on Machine Learning Research (01/2024)

DINOv2: Learning Robust Visual Features without Supervision

Maxime Oquab, Timothée Darcet**, Théo Moutakanni****,
Huy V. Vo*, **Marc Szafraniec***, **Vasil Khalidov***, **Pierre Fernandez, Daniel Haziza,**
Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba,
Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat,
Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal[1],
Patrick Labatut*, **Armand Joulin***, **Piotr Bojanowski***

Meta AI Research [1]*Inria*

*core team **equal contribution

Reviewed on OpenReview: https://openreview.net/forum?id=a68SUt6zFt

arXiv:2304.07193v2 [cs.CV] 2 Feb 2024

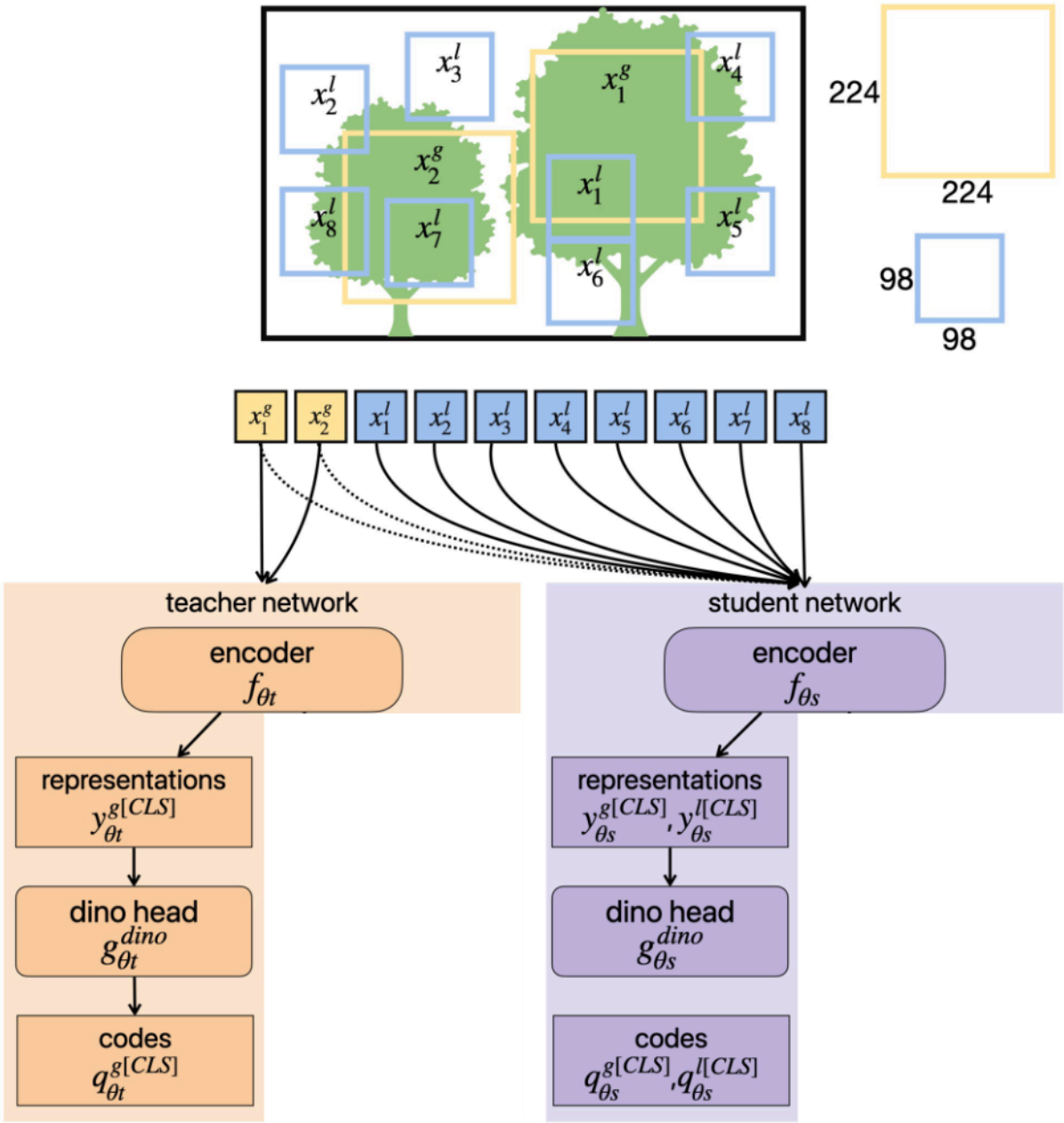
Abstract

The recent breakthroughs in natural language processing for model pretraining on large quantities of data have opened the way for similar foundation models in computer vision. These models could greatly simplify the use of images in any system by producing general-

key change: additional losses

DINO loss on <cls> token

$$\mathcal{L}_{DINO} = -\sum p_t \log p_s$$



Graphic by Mashaan Alshammari

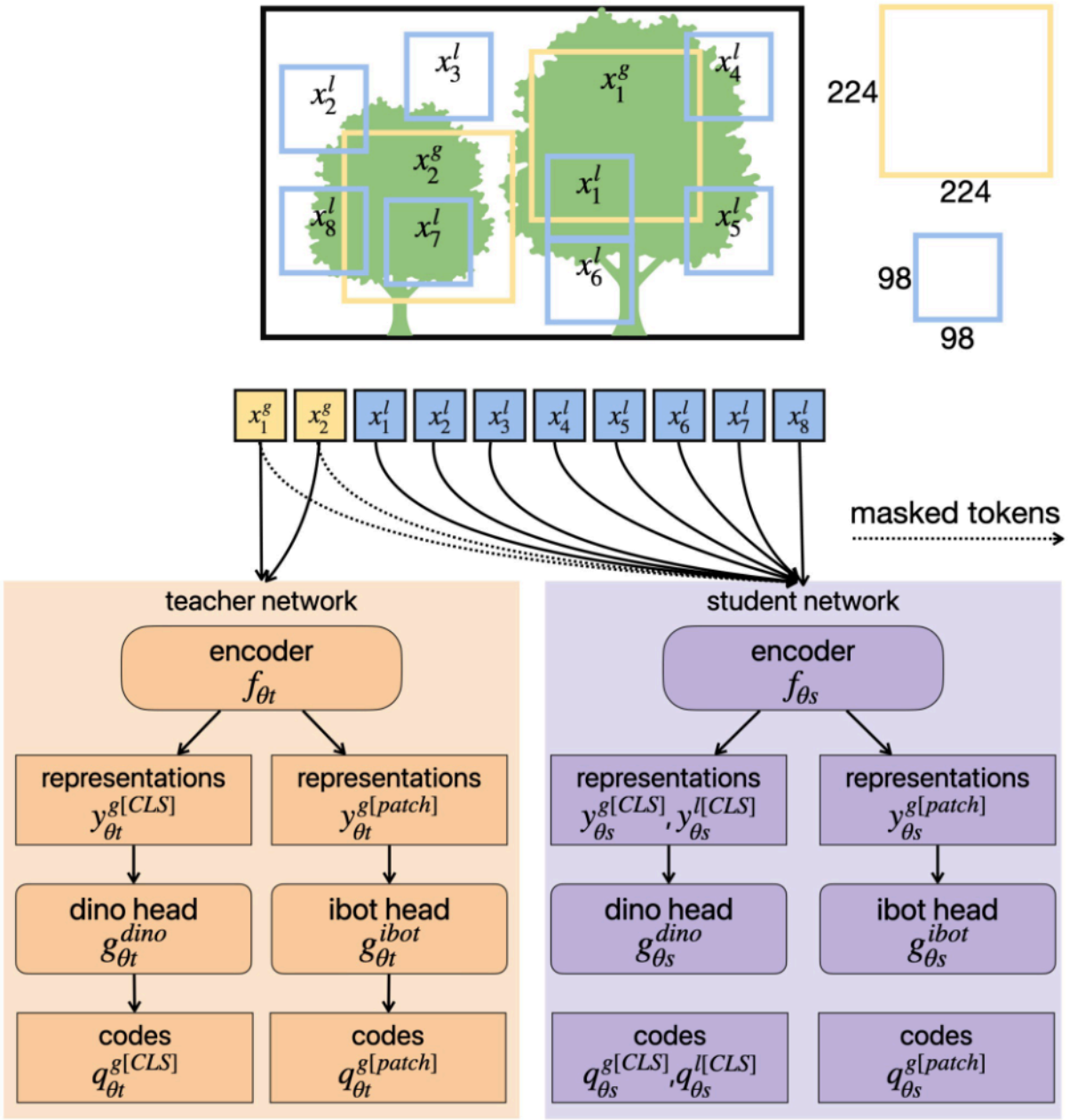
key change: additional losses

DINO loss on <cls> token

$$\mathcal{L}_{DINO} = -\sum p_t \log p_s$$

iBOT loss on masked patch tokens

$$\mathcal{L}_{iBOT} = -\sum_i p_{ti} \log p_{si}$$



Graphic by Mashaan Alshammari

key change: additional losses

DINO loss on <cls> token

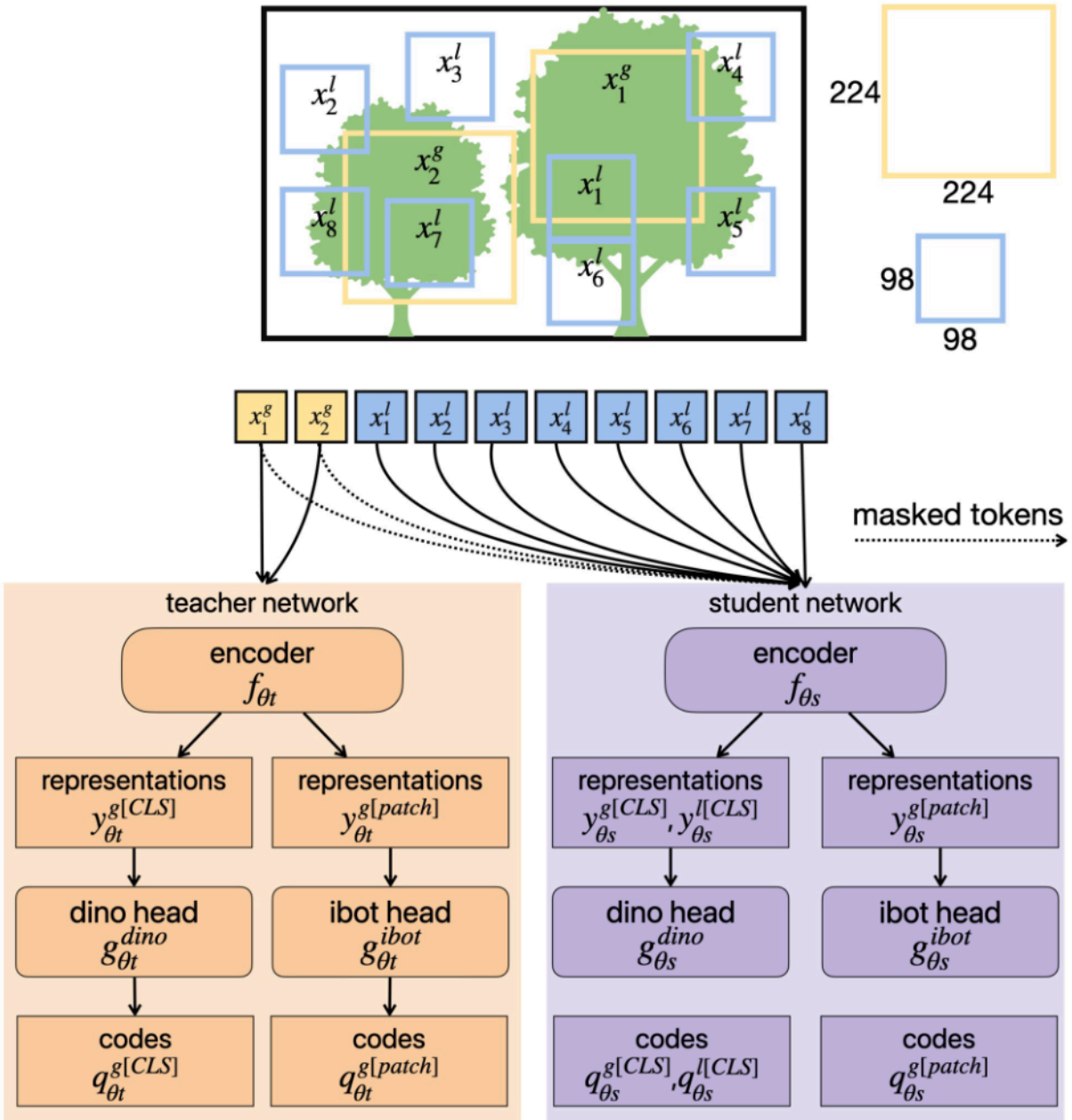
$$\mathcal{L}_{DINO} = -\sum p_t \log p_s$$

iBOT loss on masked patch tokens

$$\mathcal{L}_{iBOT} = -\sum_i p_{ti} \log p_{si}$$

koleo regularizer: encourages feature distribution in the batch

$$\mathcal{L}_{\text{koleo}} = -\frac{1}{n}\sum_{i=1}^{n} \log(d_{n,i}),$$



Graphic by Mashaan Alshammari

Method	Arch.	Data	Text sup.	kNN	linear		
				val	val	ReaL	V2
Weakly supervised							
CLIP	ViT-L/14	WIT-400M	✓	79.8	84.3	88.1	75.3
CLIP	ViT-L/14_{336}	WIT-400M	✓	80.5	85.3	88.8	75.8
SWAG	ViT-H/14	IG3.6B	✓	82.6	85.7	88.7	77.6
OpenCLIP	ViT-H/14	LAION-2B	✓	81.7	84.4	88.4	75.5
OpenCLIP	ViT-G/14	LAION-2B	✓	83.2	86.2	89.4	77.2
EVA-CLIP	ViT-g/14	custom*	✓	**83.5**	86.4	89.3	77.4
Self-supervised							
MAE	ViT-H/14	INet-1k	✗	49.4	76.6	83.3	64.8
DINO	ViT-S/8	INet-1k	✗	78.6	79.2	85.5	68.2
SEERv2	RG10B	IG2B	✗	–	79.8	–	–
MSN	ViT-L/7	INet-1k	✗	79.2	80.7	86.0	69.7
EsViT	Swin-B/W=14	INet-1k	✗	79.4	81.3	87.0	70.4
Mugs	ViT-L/16	INet-1k	✗	80.2	82.1	86.9	70.8
iBOT	ViT-L/16	INet-22k	✗	72.9	82.3	87.5	72.4
DINOv2	ViT-S/14	LVD-142M	✗	79.0	81.1	86.6	70.9
	ViT-B/14	LVD-142M	✗	82.1	84.5	88.3	75.1
	ViT-L/14	LVD-142M	✗	**83.5**	86.3	89.5	78.0
	ViT-g/14	LVD-142M	✗	**83.5**	**86.5**	**89.6**	**78.4**

Table 4: **Linear evaluation on ImageNet-1k of frozen pretrained features.** We report Top-1 accuracy on the validation set for publicly available models trained on public or private data, and with or without text supervision (text sup.). For reference, we also report the kNN performance on the validation set. We compare across any possible architectures (Arch.), at resolution 224 × 224 unless stated otherwise. The dataset used for training EVA-CLIP is a custom mixture, see paper for details (Fang et al., 2023).

well-generalizing patch-level features



Figure 8: **Examples of out-of-distribution examples** with frozen DINOv2-g features and a linear probe.

why Meta?

Published in Transactions on Machine Learning Research (01/2024)

DINOv2: Learning Robust Visual Features without Supervision

Maxime Oquab**, Timothée Darcet**, Théo Moutakanni**,
Huy V. Vo*, Marc Szafraniec*, Vasil Khalidov*, Pierre Fernandez, Daniel Haziza,
Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba,
Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat,
Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal[1],
Patrick Labatut*, Armand Joulin*, Piotr Bojanowski*

Meta AI Research [1]*Inria*

*Core team **Equal contribution

Reviewed on OpenReview: https://openreview.net/forum?id=a68SUt6zFt

arXiv:2304.07193v2 [cs.CV] 2 Feb 2024

Abstract

The recent breakthroughs in natural language processing for model pretraining on large quantities of data have opened the way for similar foundation models in computer vision. These models could greatly simplify the use of images in any system by producing general-

A main reason for SSL is scale

DINO: trained on ImageNet-1k (1.3 M images)

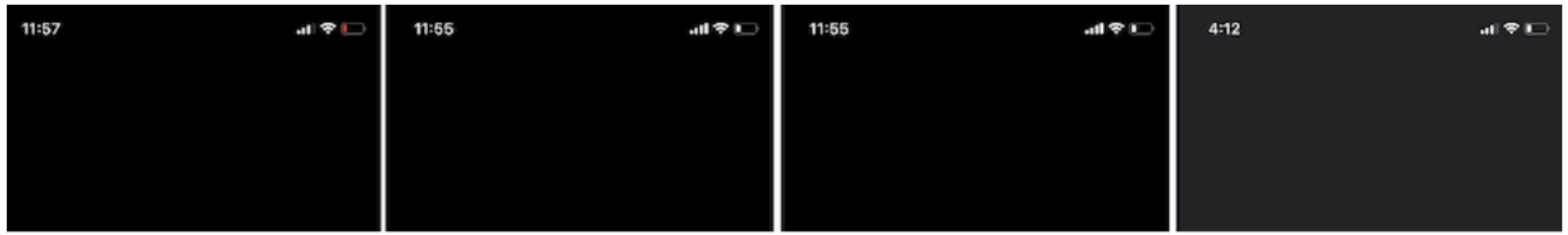
DINOv2: trained on 142 M images

DINOv3: trained on 1689 M images

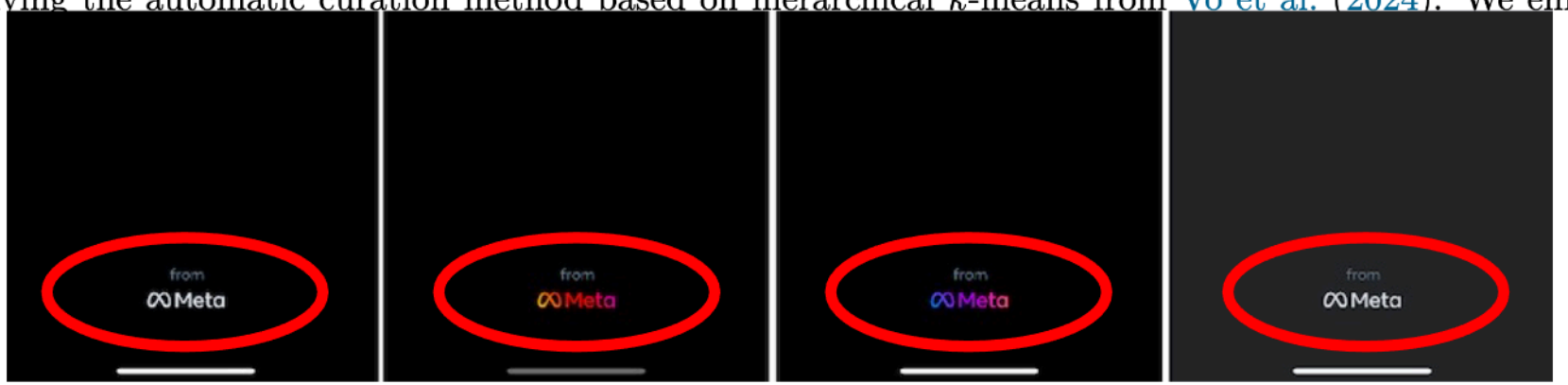
Table 3: Dense linear probing results on semantic segmentation and monocular depth estimation with frozen backbones. We report the mean Intersection-over-Union (mIoU) metric for the segmentation benchmarks ADE20k, Cityscapes, and VOC. We report the Root Mean Squared Error (RMSE) metric for the depth benchmarks NYUv2 and KITTI. For segmentation, all models are evaluated with input resolution adapted to 1024 patch tokens (*i.e.* 448 × 448 for patch size 14, 512 × 512 for patch size 16).

		Segmentation			Depth	
Method	ViT	ADE20k	Citysc.	VOC	NYUv2 ↓	KITTI ↓
Agglomerative backbones						
AM-RADIOv2.5	g/14	53.0	78.4	85.4	0.340	2.918
PEspatial	G/14	49.3	73.2	82.7	0.362	3.082
Weakly-supervised backbones						
SigLIP 2	g/16	42.7	64.8	72.7	0.494	3.273
PEcore	G/14	38.9	61.1	69.2	0.590	4.119
Self-supervised backbones						
Franca	g/14	46.3	68.7	82.9	0.445	3.140
DINOv2	g/14	49.5	75.6	83.1	0.372	2.624
Web-DINO	7B/14	42.7	68.3	76.1	0.466	3.158
DINOv3	7B/16	**55.9**	**81.1**	**86.6**	**0.309**	**2.346**

Who has 1643 M object-centric images?



Data Collection and Curation We build our large-scale pre-training dataset by leveraging a large data pool of web images collected from public posts on Instagram. These images already went through platform-level content moderation to help prevent harmful contents and we obtain an initial data pool of approximately 17 billions of images. Using this raw data pool, we create three dataset *parts*. We construct the first part by applying the automatic curation method based on hierarchical k-means from Vo et al. (2024). We employ



How can we train on so many images?

Curated Data

- license information
- no duplicates
- annotations

Crawl Data

- highly unbalanced
- no content moderation
- watermarks and other artifacts

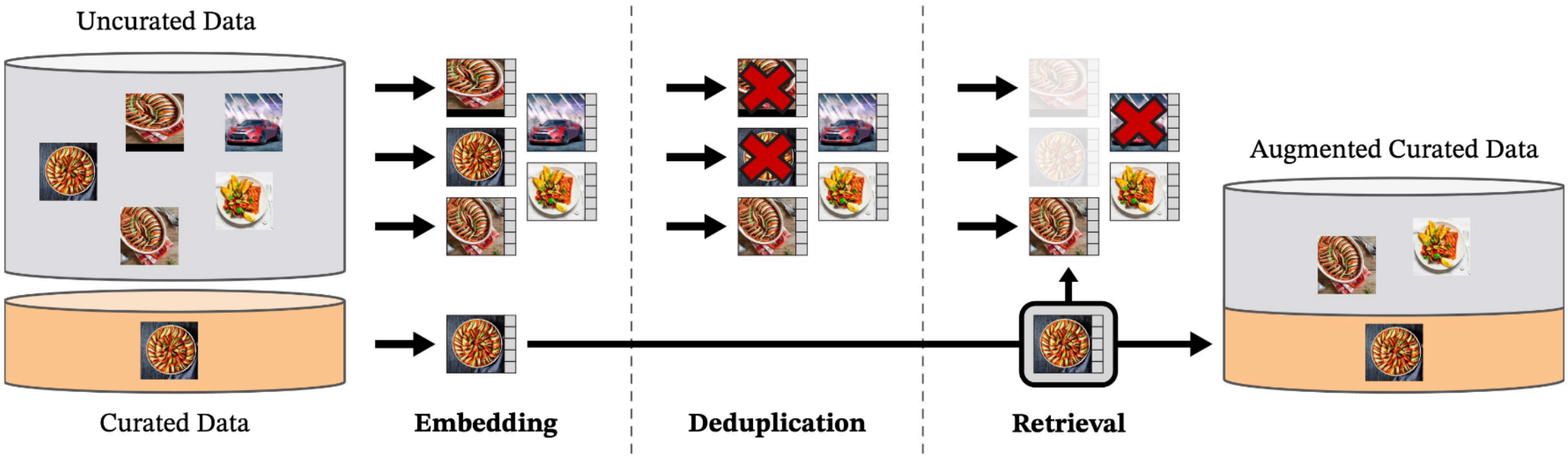
DINO v2 data processing

1. fast download postprocessing: hash-based deduplication, NSFW filters, face blurring
2. copy detection / near-duplicate removal
   + remove any images that may be near-duplicates of test data
3. image embedding + clustering (with a baseline embedding)
4. retrieval based on distance to curated datasets
   -> for every image in ImageNet, Google Landmarks, etc this yields 4 similar web-crawl images



source: Oquab et al, DINO v2

DINO v2 data processing



source: Oquab et al, DINO v2

DINO v2 data processing

1. fast download postprocessing: has
   deduplication, NSFW filters, face b
2. copy detection / near-duplicate rer
   + remove any images that may b
   of test data
3. image embedding + clustering (with a baseline embedding)
4. retrieval based on distance to curated datasets -> for every image in ImageNet, Google Landmarks, etc this yields 4 similar web-crawl images

Why k-means clustering?



DINO v3 data processing

Part A

1. Instagram comes with content moderation
2. hierarchical k-means with DINOv2

17 B

1.7 B

Part B

3. kNN retrieval into Part A from curated „seed“ datasets

Part C

ImageNet + Mapillary Street Sequences

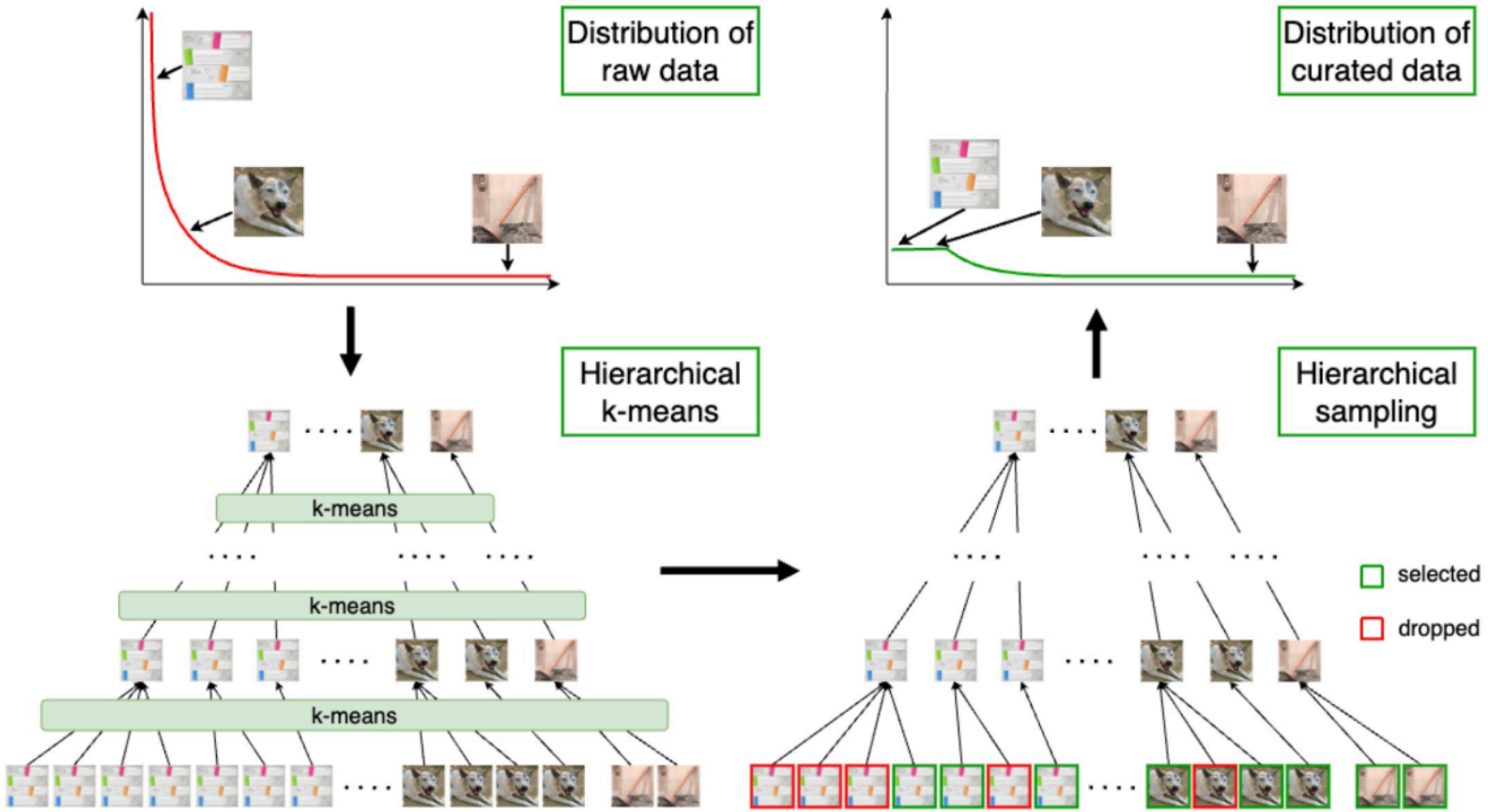


16 M

At training, 10% of batches are ImageNet, 90% heterogenous mixes of all parts

source: Siméoni et al, DINO v3

hierarchical k-means for data balancing



source: Vo et al, 2024

hierarchical k-means for data balancing

Algorithm 1: Hierarchical k-means with resampling algorithm.

Input: Data $X \in \mathbb{R}^{n \times d}$, number of levels T, number of clusters per level $(k_t)_{1 \leq t \leq T}$, number of resamplings m, number of points resampled per cluster $(r_t)_{1 \leq t \leq T}$.

Result: A hierarchy of clusters on data: centroids $(C_t)_{1 \leq t \leq T}$ and clusters $((L_t^{(i)})_{1 \leq i \leq k_t})_{1 \leq t \leq T}$.

for $t = 1$ to T **do**
  if $t = 1$ **then** $\mathbf{I} \leftarrow X$ **else** $\mathbf{I} \leftarrow C_{t-1}$ ▷ Get input $\mathbf{I}$ of level t
  $C_t \leftarrow \texttt{kmeans}(\mathbf{I}, k_t)$ ▷ Find centroids C_t with k-means
  $L_t \leftarrow \texttt{assign}(\mathbf{I}, C_t)$ ▷ Assign clusters $(L_t^{(i)})_{1 \leq i \leq n_t}$ with k-means
  # resampling-clustering
  for $s = 1$ to m **do**
    $R \leftarrow \bigcup_{i=1}^{k_t} \texttt{resample}(L_t^{(i)}, r_t)$ ▷ Sample r_t points from each cluster
    $C_t \leftarrow \texttt{kmeans}(R, k_t)$ ▷ Find centroids based on resampled set
    $L_t \leftarrow \texttt{assign}(\mathbf{I}, C_t)$ ▷ Assign clusters to entire input set of level t
  end
end

source: Vo et al, 2024

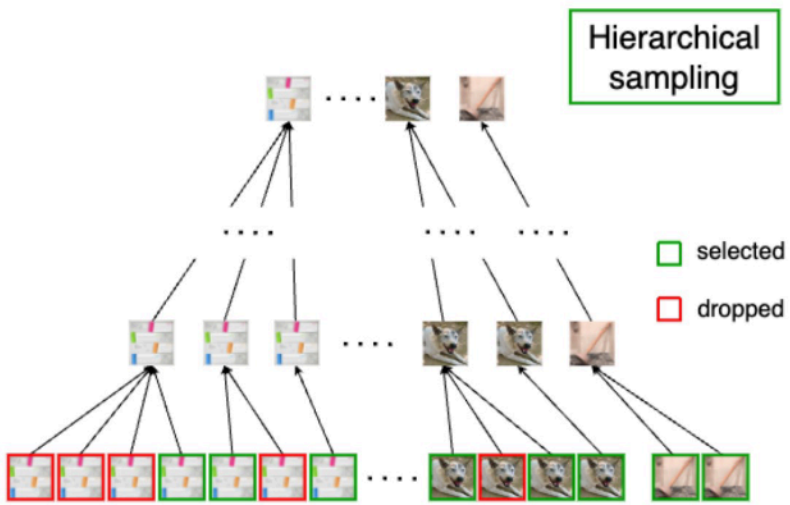
sampling from hierarchical clusters

1. compute size s_j of each subtree bottom-up
2. balance samples n per subtree as

$$|N - \sum_j^k \min(n, s_j)|$$

   where N is the number of samples over all subtrees

3. go one level down, until you reach the leafes



repeat this process down to level 1. Once we have these numbers for all clusters in level 1, we sample from them to form the balanced subset. This strategy, named hierarchical sampling, guarantees a balance among concepts represented by the sub-trees at the highest levels (such as animal, vehicle, sport, etc.) and among sub-concepts represented by internal nodes in lower levels (such as dog, spider, airplane, scooter, football, vovinam, etc.). We consider several ways to sample data points from clusters of level 1 such as sampling random points ("r"), or points that are closest ("c") or furthest ("f") from their centroids. We compare the

source: Vo et al, 2024

Summary

SiamSiam: Negatives are not necessary

BERT

DINO: Multi-crop emerges as useful general augmentation

scaling requires data engines

